

Java - Collections and Strings

Questions and Answers

Name: _____ Roll No: _____ Date: _____

A. Java Collection Framework (35 Questions)

Q1. What is the Java Collection Framework?

The Java Collection Framework (JCF) is a unified architecture in the `java.util` package for storing, retrieving and manipulating groups of objects. It provides:

- **Interfaces** such as Collection, List, Set, Queue, Deque and Map.
- **Implementations** (ready-made classes) such as ArrayList, LinkedList, HashSet, TreeSet, PriorityQueue, HashMap and TreeMap.
- **Algorithms** (static methods in the Collections class) for sorting, searching, reversing, shuffling, etc.

Hierarchy: Iterable -> Collection -> List / Set / Queue. Map is a separate hierarchy and does not extend Collection.

Q2. Why do we use collections instead of arrays?

- Arrays have a **fixed size**; collections are **dynamic** and grow or shrink automatically.
- Arrays have no built-in methods for add, remove, search or sort; collections provide them.
- Collections offer different data structures (list, set, queue, map) for different needs.
- Collections store objects only (primitives are auto-boxed) and support generics for type safety.
- Arrays can hold primitives and are slightly faster and lighter when the size is known in advance.

Q3. What is the difference between Collection and Collections?

- **Collection** is an **interface** (`java.util.Collection`), the root of List, Set and Queue.
- **Collections** is a **utility class** with static helper methods such as `sort()`, `reverse()`, `shuffle()`, `max()`, `min()`, `binarySearch()`, `unmodifiableList()` and `synchronizedList()`.

Q4. What is the difference between Collection and Map?

- Collection stores a group of **single objects**; Map stores **key-value pairs**.
- Map does **not** extend the Collection interface.
- Collection uses `add()`; Map uses `put(key, value)`.
- A Map cannot have duplicate keys; List (a Collection) can have duplicate elements.

Q5. What is the difference between List, Set, and Queue?

- **List**: ordered (insertion order), index-based access, allows duplicates. Example: ArrayList, LinkedList.
- **Set**: no duplicates, no index-based access. Example: HashSet, LinkedHashSet, TreeSet.
- **Queue**: holds elements for processing, usually in FIFO (First-In-First-Out) order. Example: LinkedList, PriorityQueue, ArrayDeque.

Q6. What is the difference between ArrayList and LinkedList?

- **ArrayList** uses a dynamic (resizable) array internally; **LinkedList** uses a doubly linked list.
- ArrayList: fast random access `get(i)` in $O(1)$; slow insertion or deletion in the middle, $O(n)$ due to shifting.
- LinkedList: fast insertion or deletion at the ends or when the node is known, $O(1)$; slow `get(i)`, $O(n)$.
- LinkedList also implements Deque/Queue; it uses more memory (each node stores two references).
- Use ArrayList for frequent reading; LinkedList for frequent insert or delete.

Q7. What are the important features of ArrayList?

- Resizable array; default initial capacity is 10 and it grows by about 50% when full.
- Maintains insertion order and allows duplicate elements.
- Allows null values.
- Supports fast random access through the index (implements RandomAccess).
- Not synchronized (not thread-safe); use `Collections.synchronizedList()` or `CopyOnWriteArrayList` for threads.
- Can hold only objects (wrapper classes for primitives).

Q8. How do you add elements to an ArrayList?

Use `add(element)` to append at the end, `add(index, element)` to insert at a position, and `addAll(collection)` to add many.

```
ArrayList<String> list = new ArrayList<>();  
list.add("Java");           // [Java]  
list.add("Python");         // [Java, Python]  
list.add(1, "C++");         // [Java, C++, Python]
```

Q9. What is the difference between add() and add(index, element)?

- `add(e)` appends the element at the **end** of the list and returns boolean (true).
- `add(index, e)` **inserts** the element at the given position, shifting existing elements to the right; returns void.
- If the index is out of range (`index < 0` or `index > size`), `add(index, e)` throws `IndexOutOfBoundsException`.

Q10. How do you access an element from an ArrayList?

Use `get(index)`. Index starts at 0. An invalid index throws `IndexOutOfBoundsException`.

```
String s = list.get(0);    // "Java"
```

Q11. How do you remove an element from an ArrayList?

- `remove(int index)` removes the element at that position.
- `remove(Object o)` removes the first occurrence of that element.
- `clear()` removes all elements; `removeIf(condition)` removes by condition; `removeAll(collection)` removes many.

```
list.remove(0);            // by index  
list.remove("Python");     // by object
```

Q12. What is the difference between remove(int index) and remove(Object)?

- `remove(int index)` removes by **position** and **returns the removed element**.
- `remove(Object o)` removes the **first matching element** and **returns boolean** (true if found).

Example: in a `List<Integer>`, `list.remove(1)` removes the element at index 1. To remove the value 1, use `list.remove(Integer.valueOf(1))`.

Q13. How do you find the size of an ArrayList?

Use the `size()` method. (Arrays use the `length` property and Strings use `length()`.)

```
int n = list.size();
```

Q14. How do you check whether an ArrayList contains a particular element?

Use `contains(Object)`, which returns true or false. `indexOf(Object)` returns the position, or -1 if not found. Matching uses `equals()`.

```
boolean found = list.contains("Java");    // true
```

Q15. How do you iterate through an ArrayList?

There are several ways: simple for loop, enhanced for-each loop, Iterator, ListIterator (both directions), `forEach()` with a lambda, and streams.

```
for (int i = 0; i < list.size(); i++) System.out.println(list.get(i));
for (String s : list) System.out.println(s);
Iterator<String> it = list.iterator();
while (it.hasNext()) System.out.println(it.next());
list.forEach(s -> System.out.println(s));
```

Q16. What is a Set?

A Set is a Collection that **does not allow duplicate elements**. It models the mathematical set. Main implementations: HashSet (unordered), LinkedHashSet (insertion order) and TreeSet (sorted). It has no index-based access.

Q17. What is the main difference between List and Set?

- List allows duplicates; Set does not.
- List maintains order and supports index access (`get(i)`); Set generally has no index access.
- List allows many nulls; Set allows at most one null (HashSet, LinkedHashSet) and TreeSet allows none.

Q18. Does a HashSet allow duplicate elements?

No. If you add a duplicate, `add()` returns false and the set is unchanged. Duplicates are detected using `hashCode()` and `equals()`.

Q19. Does a HashSet maintain insertion order?

No. HashSet stores elements by hash code (backed by a HashMap), so iteration order is not guaranteed. Use LinkedHashSet to keep insertion order, or TreeSet for sorted order.

Q20. What is the difference between HashSet and LinkedHashSet?

- HashSet: no ordering guarantee; slightly faster and uses less memory.
- LinkedHashSet: maintains **insertion order** using a doubly linked list over the hash table; slightly slower.
- Both are unique-only, allow one null, are not synchronized, and have O(1) average add/remove/contains.

Q21. What is the difference between HashSet and TreeSet?

- HashSet: unordered, uses hash table, O(1) average operations, allows one null.
- TreeSet: **sorted** (natural order or Comparator), uses a red-black tree, O(log n) operations, does **not** allow null (throws NullPointerException).
- TreeSet elements must be comparable (Comparable) or a Comparator must be supplied.

Q22. What is the purpose of TreeSet?

TreeSet stores **unique elements in sorted order**. It also provides navigation methods like `first()`, `last()`, `floor()`, `ceiling()`, `higher()`, `lower()`, `headSet()`, `tailSet()` and `subSet()`. Use it when you need a sorted, duplicate-free collection.

Q23. How does TreeSet maintain its elements?

TreeSet is backed by a **TreeMap**, which is a self-balancing **red-black tree**. On every insert, the element is compared with existing ones using `compareTo()` (Comparable) or a supplied **Comparator**, and placed at the correct position. This keeps elements sorted, and two elements that compare as equal are treated as duplicates.

```
TreeSet<Integer> ts = new TreeSet<>(Arrays.asList(30, 10, 20, 10));
System.out.println(ts); // [10, 20, 30]
```

Q24. What is a Queue?

A Queue is a Collection used to hold elements before processing, normally following **FIFO** (First-In-First-Out): elements are added at the tail and removed from the head. Implementations: `LinkedList`, `ArrayDeque` and `PriorityQueue` (ordered by priority). Its core methods: `offer/add`, `poll/remove`, `peek/element`.

Q25. What is the difference between add() and offer() in a Queue?

- Both insert an element at the tail.
- `add()` throws `IllegalStateException` if the queue is capacity-restricted and full.
- `offer()` returns **false** instead of throwing an exception. It is preferred for bounded queues.

Q26. What is the difference between poll() and remove() in a Queue?

- Both remove and return the head of the queue.
- `poll()` returns **null** if the queue is empty.
- `remove()` throws `NoSuchElementException` if the queue is empty.

Q27. What is the difference between peek() and element()?

- Both return the head **without removing** it.
- `peek()` returns **null** if the queue is empty.
- `element()` throws `NoSuchElementException` if the queue is empty.

```
Queue<String> q = new LinkedList<>();
q.offer("A"); q.offer("B");
q.peek();    // A (not removed)
q.poll();    // A (removed)
q.poll();    // B
q.poll();    // null (empty)
```

Q28. What is a Map in Java?

A Map is an interface in `java.util` that stores data as **key-value pairs**. Keys are unique; each key maps to one value. Main implementations: `HashMap`, `LinkedHashMap`, `TreeMap`, `Hashtable`. Common methods: `put()`, `get()`, `remove()`, `containsKey()`, `containsValue()`, `size()`, `keySet()`, `values()`, `entrySet()`.

```
Map<Integer, String> m = new HashMap<>();
m.put(1, "Java");
m.put(2, "Python");
System.out.println(m.get(1)); // Java
```

Q29. What is the difference between a Map and a Set?

- Map stores key-value pairs; Set stores single elements.
- Map keys are unique but values can repeat; every Set element is unique.
- Map does not extend `Collection`; Set does.
- Internally, a `HashSet` is implemented using a `HashMap` (elements are stored as keys).

Q30. Does a HashMap allow duplicate keys?

No. If you put () with an existing key, the old value is **replaced** by the new one (and the old value is returned). HashMap allows one null key.

Q31. Can a HashMap contain duplicate values?

Yes. Different keys can map to the same value. HashMap also allows multiple null values.

Q32. What is the difference between HashMap and LinkedHashMap?

- HashMap: no order guaranteed; slightly faster.
- LinkedHashMap: maintains **insertion order** (or access order if configured) using a doubly linked list; slightly more memory.
- Both allow one null key and multiple null values, and are not synchronized.

Q33. What is the difference between HashMap and TreeMap?

- HashMap: unordered, hash table, O(1) average operations, allows one null key.
- TreeMap: keys kept **sorted** (natural order or Comparator), red-black tree, O(log n) operations, **no null key** (null values allowed).
- TreeMap provides navigation methods like firstKey(), lastKey(), headMap(), tailMap().

Q34. What is the purpose of keySet(), values(), and entrySet()?

- keySet () returns a **Set** of all keys.
- values () returns a **Collection** of all values.
- entrySet () returns a **Set of Map.Entry** objects (each with getKey () and getValue ()); it is the best way to iterate over keys and values together.

```
for (Map.Entry<Integer, String> e : m.entrySet())
    System.out.println(e.getKey() + " = " + e.getValue());
```

Q35. What is an Iterator, and why is it used with collections?

An Iterator is an object (java.util.Iterator) used to **traverse a collection one element at a time**. Methods: hasNext (), next () and remove ().

- It gives a uniform way to traverse any collection (List, Set, Queue).
- It allows **safe removal** of elements while iterating (via it.remove ()).
- Iterators are **fail-fast**: modifying the collection directly during iteration throws ConcurrentModificationException.
- Iterator moves forward only; ListIterator can move in both directions.

```
Iterator<String> it = list.iterator();
while (it.hasNext()) {
    if (it.next().equals("C++")) it.remove();    // safe
}
```

B. Strings in Java (15 Questions)

Q36. What is a String in Java?

A String is a sequence of characters, represented by the **final class** java.lang.String. Strings are **objects**, not primitive types. They are immutable and can be created with a literal (String s = "Hello";) or with new String("Hello").

Q37. Why is String immutable in Java?

Immutable means that once created, its value cannot be changed (any change creates a new object). Reasons:

- **String pool:** many references can safely share one literal, which saves memory.
- **Security:** Strings are used for passwords, file paths, URLs and class names; they must not change after validation.
- **Thread safety:** immutable objects can be shared across threads without synchronization.
- **Hashcode caching:** the hash code is computed once, which makes Strings efficient HashMap keys.

Q38. What is the difference between String and StringBuilder?

- String is **immutable**; StringBuilder is **mutable** (`append()`, `insert()`, `delete()`, `reverse()` modify the same object).
- Repeated concatenation with String creates many temporary objects (slow); StringBuilder is much faster for that.
- String can be in the String pool; StringBuilder objects live only on the heap.

Q39. What is the difference between StringBuilder and StringBuffer?

- Both are mutable character sequences with the same methods.
- **StringBuffer** is **synchronized** (thread-safe) but slower.
- **StringBuilder** is **not synchronized**, so it is faster (introduced in Java 5). Use it when thread safety is not needed.

Q40. What is the difference between == and equals() when comparing Strings?

- `==` compares **references** (whether both variables point to the same object).
- `equals()` compares the **content** (characters) of the Strings.

```
String a = "Java";
String b = "Java";
String c = new String("Java");
a == b;           // true  (same pool object)
a == c;           // false (different objects)
a.equals(c);      // true  (same content)
```

Q41. What is the String pool?

The String pool (String constant pool) is a special memory area inside the **heap** where String literals are stored. When you write a literal, the JVM checks the pool: if it exists, the same reference is reused; otherwise a new String is created in the pool. `new String("x")` always creates a new object in the heap, and `intern()` returns the pooled version. This saves memory.

Q42. What is the use of the length() method?

`length()` returns the **number of characters** in the String (an int).

```
"Hello".length();    // 5
```

Q43. What is the use of the charAt() method?

`charAt(int index)` returns the **character at the given index** (starting at 0). An invalid index throws `StringIndexOutOfBoundsException`.

```
"Hello".charAt(1);    // 'e'
```

Q44. What is the use of the substring() method?

It extracts a part of the String.

- `substring(begin)` returns from begin to the end.
- `substring(begin, end)` returns from begin up to **end - 1** (end index is exclusive).

```
String s = "Hello World";
s.substring(6);      // "World"
s.substring(0, 5);   // "Hello"
```

Q45. What is the difference between equals() and equalsIgnoreCase()?

- `equals()` is **case-sensitive**: "Java" and "java" are not equal.
- `equalsIgnoreCase()` ignores case differences: "Java" and "JAVA" are equal.

```
"Java".equals("java");      // false
"Java".equalsIgnoreCase("java"); // true
```

Q46. What is the use of contains()?

`contains(CharSequence s)` checks whether the String contains the given sequence and returns **true** or **false**. It is case-sensitive.

```
"Hello World".contains("World"); // true
```

Q47. What is the use of indexOf() and lastIndexOf()?

- `indexOf(x)` returns the index of the **first** occurrence of a character or substring, or **-1** if not found.
- `lastIndexOf(x)` returns the index of the **last** occurrence, or **-1** if not found.

```
String s = "banana";
s.indexOf("a");      // 1
s.lastIndexOf("a");  // 5
s.indexOf("z");      // -1
```

Q48. What is the difference between replace() and replaceAll()?

- `replace(old, new)` replaces **all occurrences** of a literal character or sequence (no regex).
- `replaceAll(regex, replacement)` replaces all matches of a **regular expression**.
- `replaceFirst(regex, replacement)` replaces only the first match.

Both return a new String; the original is unchanged.

```
"a.b.c".replace(".", "-");      // "a-b-c"
"a1b22c".replaceAll("[0-9]", ""); // "abc"
```

Q49. What is the use of trim()?

`trim()` removes **leading and trailing whitespace** (characters up to U+0020) and returns a new String. It does not remove spaces in the middle. From Java 11, `strip()` is the Unicode-aware alternative.

```
" Hello ".trim(); // "Hello"
```

Q50. How would you reverse a String in Java?

The simplest way is `StringBuilder.reverse()`. Other ways: a loop that builds the string from the end, or swapping characters in a char array.

```
// 1. StringBuilder
String rev = new StringBuilder("Hello").reverse().toString();    // "olleH"

// 2. Loop
String s = "Hello", r = "";
for (int i = s.length() - 1; i >= 0; i--) r += s.charAt(i);

// 3. Char array (two pointers)
char[] ch = s.toCharArray();
for (int i = 0, j = ch.length - 1; i < j; i++, j--) {
    char t = ch[i]; ch[i] = ch[j]; ch[j] = t;
}
String r2 = new String(ch);
```